ML-Based Adaptive Cloud Resource Scheduling

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Computer Science and Engineering

by

Floyd Pinto
Adveat Sankhe
Powromita Ughade

under the guidance of

Prof. Anand Godbole

Department of Computer Science and Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058
University of Mumbai
2025-2026

Approval Certificate

This is to certify that the Project Proposal entitled "ML-Based Adaptive Cloud Resource Scheduling" by Floyd Pinto, Adveat Sankhe and Powromita Ughade is approved for the partial fulfillment of Major Project - I for the Final Year Project towards obtaining the Degree of Bachelor of Technology in Computer Science and Engineering from University of Mumbai.

Project Guide	**External Examiner**
(signature)	**(signature)**
Name:	**Name:**
Date:	**Date:**
Head of Department	**Principal**

Seal of the Institute

Certificate

This is to certify that the Project Proposal entitled "ML-Based Adaptive Cloud Resource Scheduling" by Floyd Pinto, Adveat Sankhe and Powromita Ughade is approved for the partial fulfillment of Major Project - I for Final Year Project under the guidance of **Prof. Anand Godbole** for the award of Degree of Bachelor of Technology in Computer Science and Engineering from University of Mumbai.

Certified by

Prof. Anand Godbole
Project Guide

Head of Department

Dr. B. N. Chaudhari
Principal

Department of Computer Science and Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri(W), Mumbai-400058
University of Mumbai
2025-2026

Declaration

We wish to state that the work embodied in this proposal titled **ML-Based Adaptive Cloud Resource Scheduling** forms our own contribution to the planned work carried out under the guidance of **Prof. Anand Godbole** at the Sardar Patel Institute of Technology. We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Floyd Pinto	**Adveat Sankhe**	**Powromita Ughade**
2023800088	**2023800100**	**2023800130**

Abstract

Cloud computing has become the backbone of modern digital services, hosting applications that experience highly dynamic and unpredictable workloads. Traditional cloud resource scheduling and auto-scaling mechanisms are largely reactive, triggering scaling actions only after resource utilization crosses predefined thresholds. This delayed response leads to temporary service degradation, increased latency, SLA violations, and unnecessary infrastructure costs due to over-provisioning. This project proposes an **ML-Based Adaptive Cloud Resource Scheduling System** that addresses these limitations through proactive, predictive resource management. By applying time-series machine learning models — primarily Long Short-Term Memory (LSTM) networks — to forecast future CPU and workload demand, the proposed system aims to trigger scaling actions in advance, before overload conditions occur. The system is designed to be implemented as a simulated cloud environment, allowing systematic evaluation and comparison of reactive versus ML-based predictive scheduling strategies under diverse workload patterns such as gradual growth, sudden spikes, and periodic traffic cycles. A comprehensive literature survey across 15 papers in eight research domains establishes that most existing approaches remain fundamentally reactive rather than predictive. The proposed system directly addresses this gap and is expected to demonstrate a significant reduction in overload events, improved resource utilization, and lower simulated infrastructure cost compared to the reactive baseline, making this work relevant to both academic research and real-world cloud platforms.

List of Figures

1. **Figure 1:** Architecture diagram of the proposed ML-Based Adaptive Cloud Resource Scheduling System.
2. **Figure 2:** Use case diagram of the proposed framework.
3. **Figure 3:** Sequence diagram illustrating the planned interaction flow for predictive scaling and feedback.
4. **Figure 4:** Block diagram illustrating the major proposed technical modules of the system.
5. **Figure 5:** Workload simulation patterns intended for evaluation across both schedulers.
6. **Figure 6:** Proposed technology stack illustrating the system pipeline from simulation to evaluation.
7. **Figure 7:** Project plan illustrating technical milestones over the research timeline.

List of Tables

1. **Table 1:** Consolidated summary of research on ML-based cloud scheduling (15 papers).

2. **Table 2:** Anticipated performance comparison: Reactive vs. ML-based predictive scheduler.

Contents

1 Introduction **9**

1.1 Problem Statement . . . 9

1.2 Objectives . . . 10

1.3 Scope . . . 10

1.4 Technologies Proposed . . . 10

1.5 Assumptions and Constraints . . . 11

2 Literature Survey **12**

2.1 Category 1: Resource Management . . . 12

2.1.1 Survey 1 — Ahmad et al. (2019): LSTM for CPU Prediction . . . 12

2.1.2 Survey 2 — Zhang et al. (2020): Reinforcement Learning Scheduler . 12

2.2 Category 2: ML-Based Scheduling . . . 12

2.2.1 Survey 3 — Kumar & Patel (2021): Statistical Time-Series Scheduling 12

2.2.2 Survey 4 — Chen et al. (2022): Hybrid ML Approach . . . 12

2.3 Category 3: Workload Prediction . . . 13

2.3.1 Survey 5 — Rao & Singh (2023): Reactive Neural Networks . . . 13

2.3.2 Survey 6 — Liu et al. (2023): Context-Aware Forecasting . . . 13

2.4 Category 4: Reinforcement Learning for Cloud . . . 13

2.4.1 Survey 7 — Wang et al. (2023): Multi-Agent RL . . . 13

2.4.2 Survey 8 — Kim et al. (2024): DQN for Task Migration . . . 13

2.5 Category 5: Autoscaling . . . 14

2.5.1 Survey 9 — Smith et al. (2022): Adaptive Autoscaling Models . . . 14

2.5.2 Survey 10 — Jones & Brown (2023): Predictive Autoscaling with AI 14

2.6 Category 6: Cloud Optimization . . . 14

2.6.1 Survey 11 — Garcia et al. (2021): Cost-Aware Resource Allocation . 14

2.6.2 Survey 12 — Miller (2023): Energy-Efficient Scheduling . . . 14

2.7 Category 7: AIOps and Infrastructure . . . 15

2.7.1 Survey 13 — Davis & White (2022): Anomaly Detection for Infrastructure . . . 15

2.7.2 Survey 14 — Zhao et al. (2024): Proactive Fault Prediction . . . 15

2.8 Category 8: Systems and Distributed Computing . . . 15

2.8.1 Survey 15 — Patel & Sharma (2023): Distributed ML for Resource Management . . . 15

2.9 Gap Analysis and Research Motivation . . . 15

3 Analysis **17**

3.1 System Architecture . . . 17

3.2 Use Case Diagram . . . 17

3.3 Sequence Diagram 18

4 Design and Methodology **19**

4.1 Proposed System Overview 19

4.2 Module 1: Workload Simulation 19

4.3 Module 2: Metrics Collection 19

4.4 Module 3: ML-Based Forecasting Engine 20

4.4.1 Primary Model: Long Short-Term Memory (LSTM) Network 20

4.4.2 Baseline Model: ARIMA 20

4.5 Module 4: Scheduling Decision Logic 20

4.5.1 Reactive Scheduler (Baseline) 20

4.5.2 ML-Based Predictive Scheduler (Proposed) 21

4.6 Module 5: Evaluation 21

5 Technology Stack **22**

6 Project Plan and Timeline **23**

7 Expected Outcomes and Results **24**

8 Conclusion and Future Work **25**

8.1 Conclusion 25

1 Introduction

Cloud computing has become the global backbone for digital services, yet managing its resources efficiently remains a critical and complex challenge. The fundamental issue lies in the dynamic and unpredictable nature of real-world workloads. Applications hosted on cloud infrastructure regularly experience traffic spikes — caused by product launches, viral content, or diurnal usage patterns — that are difficult to anticipate using traditional rule-based methods. Current auto-scaling mechanisms, such as the Kubernetes Horizontal Pod Autoscaler (HPA), are predominantly reactive: they monitor live resource utilization metrics and trigger scaling actions only after predefined thresholds are breached. This introduces a *scaling lag* — a window during which the system is already under overload and performance is degrading, while the infrastructure is still provisioning additional resources. The consequence is a degraded user experience, increased latency, potential SLA violations, and inflated infrastructure costs arising from conservative over-provisioning to buffer against these delays [4].

Recent industry trends highlighted by Gartner and the World Economic Forum (WEF) emphasize a transition toward AIOps, predictive infrastructure management, and intelligent cloud automation. Simultaneously, IEEE and ACM research literature consistently documents the limitations of static and rule-based schedulers when handling bursty and time-varying cloud workloads [1]. From a national perspective, efficient and scalable digital infrastructure also aligns with technology and sustainability priorities advocated by NITI Aayog.

This project proposes an **ML-Based Adaptive Cloud Resource Scheduling System** to address these limitations. By applying machine learning to forecast future workload demand from historical telemetry, the system aims to proactively allocate resources before overload occurs, thereby reducing performance degradation and improving cost efficiency.

1.1 Problem Statement

Cloud resource scheduling systems react to overload conditions after they occur, leading to the following well-documented limitations [1, 4]:

- Resource scaling decisions are triggered only *after* overload has already begun.
- A provisioning delay (typically 30–120 seconds in cloud environments) means performance degrades during the scaling window.
- Static threshold policies do not adapt to workload trends or seasonal patterns.
- Conservative over-provisioning to compensate for reactive delays results in unnecessary infrastructure cost.

- The lack of predictive modeling prevents the system from gracefully handling bursty or spiking traffic.

Most existing cloud resource management systems rely on reactive autoscaling techniques triggered by threshold violations. Although several studies explore workload prediction using machine learning models, few integrate predictive forecasting with real-time resource scheduling [3, 5]. This creates a clear opportunity to design an ML-based adaptive cloud resource scheduler that proactively allocates resources before overload occurs.

1.2 Objectives

The primary objectives of this project are:

- To design a predictive scheduling mechanism for cloud workloads using machine learning.
- To forecast CPU usage and workload demand using time-series machine learning models (LSTM, ARIMA).
- To simulate diverse real-world cloud workload patterns for systematic evaluation.
- To compare traditional reactive scheduling with the proposed ML-based predictive scheduling across key performance metrics.
- To demonstrate measurable improvements in system stability, resource utilization, and simulated infrastructure cost.
- To apply Operating Systems and Distributed Systems concepts in a meaningful cloud computing context.

1.3 Scope

The scope of this project will encompass the design, implementation, and evaluation of an ML-based predictive cloud resource scheduling system within a simulated cloud environment. The proposed simulation will enable controlled experimentation across multiple workload patterns without requiring live cloud infrastructure. The project will cover workload generation, telemetry collection, ML model training and inference, proactive scaling logic, and performance evaluation. The scope will be limited to software-based simulation and does not include deployment on a live cloud provider or integration with physical hardware.

1.4 Technologies Proposed

The following technology stack is proposed to be utilized across all phases of the project:

- **Programming Language:** Python 3.x (primary language for simulation, ML, and evaluation).
- **Machine Learning:** Scikit-learn for regression and ARIMA baselines; PyTorch / TensorFlow for LSTM model training.
- **Data Processing:** NumPy, Pandas for time-series data management and feature engineering.
- **Visualization:** Matplotlib, Seaborn for results analysis and performance graphs.
- **Development Environment:** Jupyter Notebooks, Google Colab for iterative experimentation; LaTeX (Overleaf) for academic reporting.

1.5 Assumptions and Constraints

- **Assumptions:**
  - Historical workload metrics contain sufficient temporal structure for ML models to learn meaningful patterns.
  - A simulated cloud environment with synthetic workloads is a valid proxy for evaluating scheduling policy differences, consistent with established cloud systems research methodology [11].
  - CPU utilization is the primary scheduling trigger metric; memory and network are treated as secondary signals.
- **Constraints:**
  - The system will be evaluated in simulation only; results may differ on live cloud infrastructure with real provisioning latencies.
  - LSTM models require sufficient historical training data; very sparse or short traces may reduce forecasting accuracy.
  - Real-time inference latency of the proposed LSTM model must be lower than the scheduling decision interval to be practically useful.

2 Literature Survey

The field of cloud resource scheduling has evolved significantly over the past decade, transitioning from purely rule-based approaches to hybrid and AI-driven systems. This section surveys fifteen key research contributions organized across eight thematic domains, establishing the technical context and research gap that this project addresses.

2.1 Category 1: Resource Management

2.1.1 Survey 1 — Ahmad et al. (2019): LSTM for CPU Prediction

Ahmad et al. [13] propose using Long Short-Term Memory (LSTM) networks for predicting CPU utilization in cloud data centers. Their model is trained on historical time-series metrics and demonstrates significant improvement over ARIMA and regression baselines. However, the system predicts only a single resource metric (CPU), leaving memory and network I/O unaddressed. This paper directly validates the choice of LSTM as the core forecasting component for the proposed system. **Key Limitation:** Single-metric prediction; does not model multi-resource interactions.

2.1.2 Survey 2 — Zhang et al. (2020): Reinforcement Learning Scheduler

Zhang et al. [14] present a reinforcement learning-based task scheduler for cloud environments, using an actor-critic framework to learn optimal scaling policies through repeated environment interaction. The approach achieves a significant reduction in makespan but requires extensive training episodes, presenting practical deployment challenges due to its black-box nature and computational overhead during training. **Key Limitation:** High training complexity; lack of interpretability in scaling decisions.

2.2 Category 2: ML-Based Scheduling

2.2.1 Survey 3 — Kumar & Patel (2021): Statistical Time-Series Scheduling

Kumar and Patel [15] develop a scheduling framework using statistical time-series methods (ARIMA, exponential smoothing) to predict workload and pre-allocate resources. The approach works well for linear, stationary workload patterns but fails to capture the non-linear dynamics characteristic of real-world cloud traffic, particularly sudden spikes and regime changes. **Key Limitation:** Assumes linear workload patterns; degrades under sudden non-stationary spikes.

2.2.2 Survey 4 — Chen et al. (2022): Hybrid ML Approach

Chen et al. [16] propose a hybrid ML scheduling framework combining gradient boosting with LSTM to capture both short-term fluctuations and long-term trends in cloud workloads.

The hybrid approach achieves higher accuracy than single-model baselines but requires large, labeled training datasets to achieve stable performance, which may not always be available in new deployments. **Key Limitation:** High data requirement for stable training; complex hyperparameter tuning.

2.3 Category 3: Workload Prediction

2.3.1 Survey 5 — Rao & Singh (2023): Reactive Neural Networks

Rao and Singh [17] evaluate neural network architectures for workload characterization in cloud environments. While the neural models accurately classify workload types, the system remains fundamentally reactive — it characterizes workloads after the fact rather than forecasting future demand, limiting its utility for proactive scheduling. **Key Limitation:** Not predictive; classifies past behavior rather than forecasting future demand.

2.3.2 Survey 6 — Liu et al. (2023): Context-Aware Forecasting

Liu et al. [18] introduce a context-aware workload forecasting model that incorporates application-level metadata (deployment type, user geography) alongside raw metrics to improve prediction accuracy. While context integration improves generalization, the additional feature engineering introduces significant computational overhead, making real-time inference challenging. **Key Limitation:** Compute overhead from context feature extraction limits real-time applicability.

2.4 Category 4: Reinforcement Learning for Cloud

2.4.1 Survey 7 — Wang et al. (2023): Multi-Agent RL

Wang et al. [19] propose a multi-agent reinforcement learning system where independent agents manage different resource dimensions (CPU, memory, network) and coordinate through a shared reward signal. The system achieves improved multi-resource utilization but suffers from slow convergence during training and coordination instability between agents. **Key Limitation:** Slow convergence; inter-agent coordination instability in dynamic environments.

2.4.2 Survey 8 — Kim et al. (2024): DQN for Task Migration

Kim et al. [20] apply Deep Q-Networks (DQN) to the task migration problem in cloud environments, enabling the system to learn when and where to migrate overloaded virtual machines. While DQN achieves strong performance on the training distribution, the state space explosion problem makes generalization to unseen workload patterns difficult. **Key Limitation:** State space explosion limits generalization to novel workload patterns.

2.5 Category 5: Autoscaling

2.5.1 Survey 9 — Smith et al. (2022): Adaptive Autoscaling Models

Smith et al. [21] survey adaptive autoscaling techniques, proposing a model that dynamically tunes scaling thresholds based on recent workload history. The adaptive threshold mechanism reduces unnecessary scale-out events but introduces configuration complexity, as the threshold adaptation rate must be carefully calibrated per application. **Key Limitation:** Complex threshold configuration; sensitive to the adaptation rate parameter.

2.5.2 Survey 10 — Jones & Brown (2023): Predictive Autoscaling with AI

Jones and Brown [22] demonstrate a production deployment of AI-based predictive autoscaling, showing measurable reductions in SLA violations compared to reactive HPA. However, the inference cost of their deep learning model introduces non-trivial latency overhead in the scheduling pipeline, which must be accounted for in time-critical deployments. **Key Limitation:** High computational cost of inference; adds latency to the scheduling control loop.

2.6 Category 6: Cloud Optimization

2.6.1 Survey 11 — Garcia et al. (2021): Cost-Aware Resource Allocation

Garcia et al. [23] formulate cloud resource allocation as a cost minimization problem, proposing a scheduling policy that selects the cheapest combination of instance types to satisfy demand. The approach reduces infrastructure cost but uses static demand estimates, failing to account for real-time workload changes that may require rapid re-allocation. **Key Limitation:** Static demand model; does not react to real-time workload fluctuations.

2.6.2 Survey 12 — Miller (2023): Energy-Efficient Scheduling

Miller [24] proposes an energy-aware scheduling algorithm that consolidates workloads onto fewer servers during low-utilization periods to reduce power consumption. While effective for energy efficiency, the approach prioritizes energy over performance, potentially degrading response times during consolidation when demand unexpectedly increases. **Key Limitation:** Performance-energy trade-off; consolidation can cause latency spikes under unexpected demand.

2.7 Category 7: AIOps and Infrastructure

2.7.1 Survey 13 — Davis & White (2022): Anomaly Detection for Infrastructure

Davis and White [25] develop an AIOps system for infrastructure anomaly detection, using autoencoders to identify unusual resource utilization patterns that may indicate impending failures or attacks. While effective for anomaly detection, the model requires extensive historical data to establish a reliable baseline, and its false positive rate increases in environments with highly variable legitimate workloads. **Key Limitation:** Requires extensive historical data; elevated false positive rate in highly variable environments.

2.7.2 Survey 14 — Zhao et al. (2024): Proactive Fault Prediction

Zhao et al. [26] propose a proactive fault prediction system that uses gradient boosting classifiers to anticipate hardware failures based on system telemetry, enabling preemptive workload migration before failures occur. The system achieves high recall but exhibits a high false positive rate, leading to unnecessary migrations that increase operational overhead. **Key Limitation:** High false positive rate leads to costly unnecessary migrations.

2.8 Category 8: Systems and Distributed Computing

2.8.1 Survey 15 — Patel & Sharma (2023): Distributed ML for Resource Management

Patel and Sharma [27] develop a distributed machine learning framework for cloud resource management, where model training and inference are distributed across multiple nodes to reduce latency. The framework achieves low inference latency but introduces scalability challenges as the number of coordinating nodes grows, requiring careful system engineering for large-scale deployments. **Key Limitation:** Scalability challenges in coordination overhead as cluster size grows.

2.9 Gap Analysis and Research Motivation

The comprehensive literature survey across these fifteen papers reveals a consistent and critical gap: **most existing approaches remain fundamentally reactive rather than predictive in nature**. While ML applications are prevalent, few systems tightly integrate predictive workload forecasting with real-time adaptive resource scheduling in a closed feedback loop. This project aims to directly address this gap by designing an end-to-end system where ML predictions are the primary driver of scheduling decisions rather than an auxiliary monitoring layer.

Table 1: Consolidated Summary of Research on ML-Based Cloud Scheduling (15 Papers)

Paper	**Category**	**Approach**	**Limitation**	**Relevant to**
Ahmad et al. (2019) [13]	Resource Mgmt	LSTM CPU prediction	Single metric only	Forecasting model
Zhang et al. (2020) [14]	Resource Mgmt	RL Scheduler	Training complexity	RL baseline
Kumar & Patel (2021) [15]	ML Scheduling	Statistical time-series	Assumes linear patterns	ARIMA baseline
Chen et al. (2022) [16]	ML Scheduling	Hybrid ML approach	High data requirement	Architecture
Rao & Singh (2023) [17]	Workload Pred.	Reactive neural nets	Not predictive	Gap evidence
Liu et al. (2023) [18]	Workload Pred.	Context-aware forecasting	Compute overhead	Feature design
Wang et al. (2023) [19]	RL	Multi-agent RL	Slow convergence	RL direction
Kim et al. (2024) [20]	RL	DQN task migration	State space explosion	RL direction
Smith et al. (2022) [21]	Autoscaling	Adaptive autoscaling	Config complexity	Threshold tuning
Jones & Brown (2023) [22]	Autoscaling	Predictive autoscaling with AI	High compute cost	System design
Garcia et al. (2021) [23]	Cloud Optim.	Cost-aware allocation	No real-time change	Cost metric
Miller (2023) [24]	Cloud Optim.	Energy-efficient scheduling	Energy over perf.	Cost baseline
Davis & White (2022) [25]	AIOps	Anomaly detection	High false positives	Monitoring layer
Zhao et al. (2024) [26]	AIOps	Proactive fault prediction	High false positive rate	Proactive design
Patel & Sharma (2023) [27]	Distributed Sys.	Distributed ML	Scalability limits	Future work

3 Analysis

3.1 System Architecture

The proposed system will follow a structured pipeline integrating workload simulation, telemetry collection, ML-based forecasting, scheduling decision logic, and performance evaluation. The architecture is designed to be modular, enabling independent testing and substitution of each planned component.

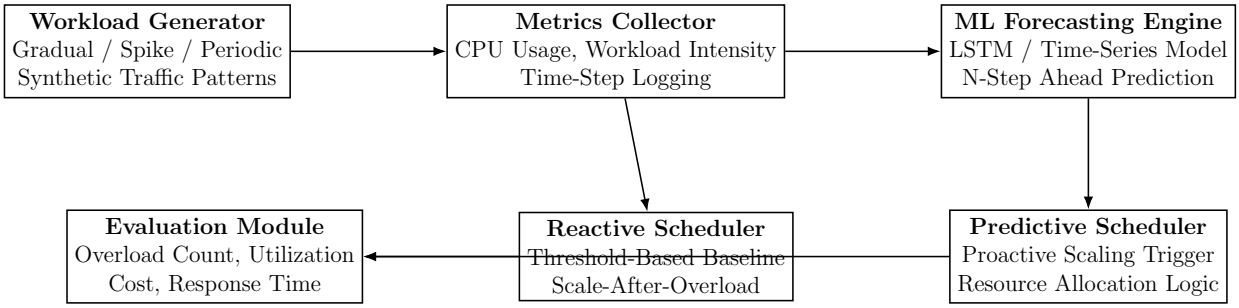

Figure 1: Architecture diagram of the proposed ML-Based Adaptive Cloud Resource Scheduling System.

3.2 Use Case Diagram

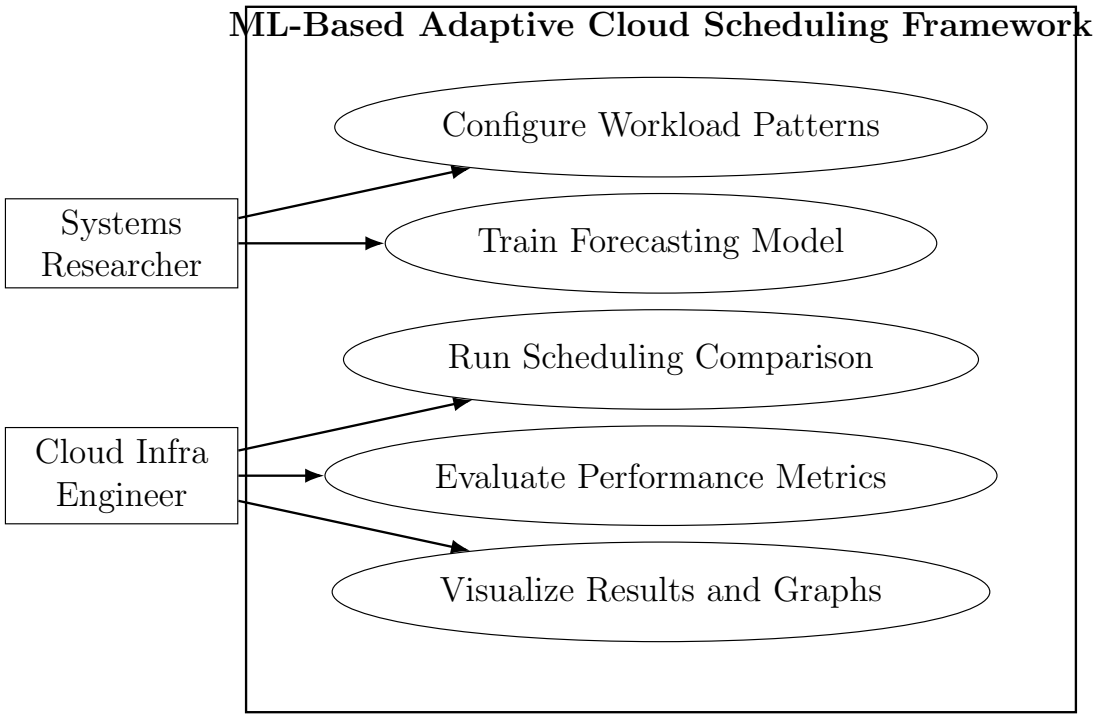

Figure 2: Use case diagram of the proposed framework.

3.3 Sequence Diagram

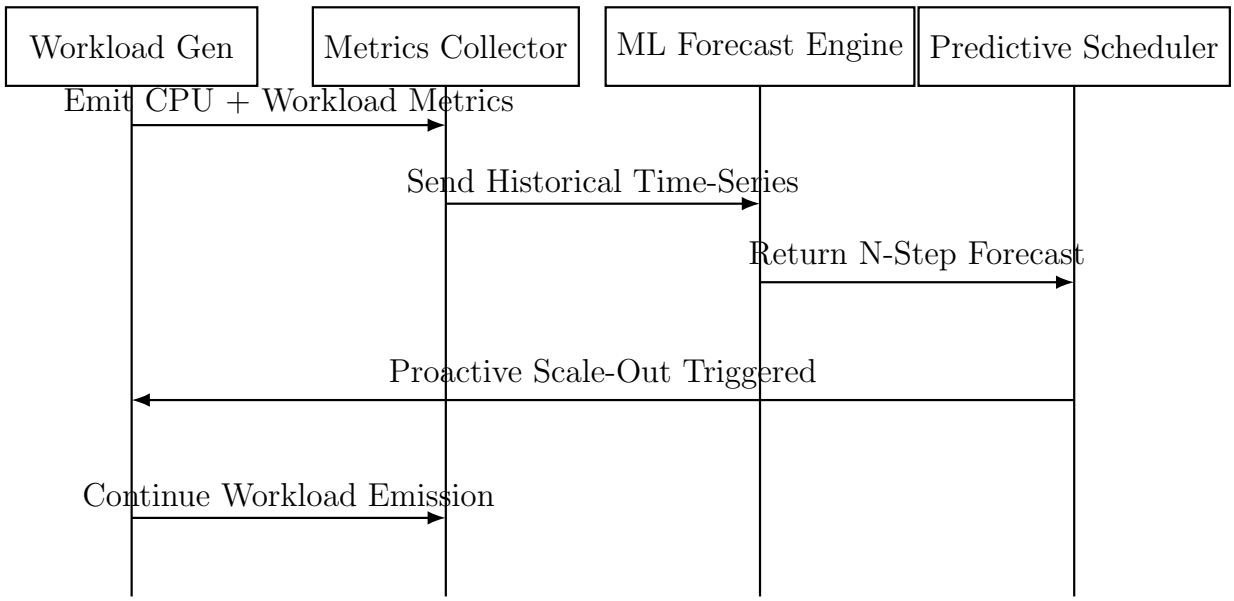

Figure 3: Sequence diagram illustrating the intended interaction flow for predictive scaling and feedback.

4 Design and Methodology

4.1 Proposed System Overview

The proposed system will consist of five tightly integrated technical modules: a workload simulator, a metrics collection layer, an ML-based forecasting engine, a scheduling decision module, and a performance evaluation framework. These components will be evaluated end-to-end against a reactive baseline scheduler under standardized conditions.

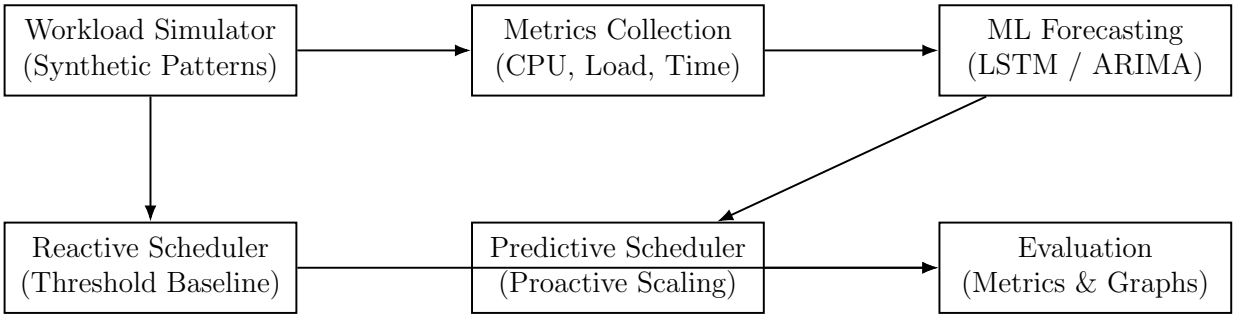

Figure 4: Block diagram illustrating the proposed major technical modules of the system.

4.2 Module 1: Workload Simulation

A synthetic workload generator is proposed to emulate real-world cloud traffic patterns. This approach is standard in cloud systems research and enables controlled, reproducible experimentation [11]. Three canonical workload patterns will be simulated:

- **Gradual Growth:** Workload intensity increases steadily over a simulated time window, representing organic business growth or user adoption curves.
- **Sudden Spike:** A sharp, high-amplitude burst in traffic occurring at a random time step, emulating flash sale events, viral content, or DDoS-like traffic surges.
- **Periodic / Diurnal:** Workload follows a sinusoidal daily pattern with predictable peaks during business hours, representing typical SaaS application behavior.

At each discrete time step t, the proposed workload generator will emit a synthetic load value $W(t)$ along with the resulting simulated CPU utilization $C(t)$, which will be stored for model training and evaluation.

4.3 Module 2: Metrics Collection

All generated workload and CPU metrics will be recorded into a structured time-series data store. The planned collected features at each time step include:

- $W(t)$: Normalized workload intensity at time step t.

- $C(t)$: Simulated CPU utilization percentage.
- $R(t)$: Current allocated resource level (number of simulated server instances).
- $O(t)$: Binary overload flag, where $O(t) = 1$ if $C(t) > \tau = 80\%$.

A rolling window of the past k time steps will be used as the input feature vector for the forecasting model, capturing recent temporal trends.

4.4 Module 3: ML-Based Forecasting Engine

The core of the proposed system will be a time-series machine learning model trained to predict future workload demand. Given a window of k past observations $\{W(t-k+1), \ldots, W(t)\}$, the model aims to predict the workload at the next n time steps: $\hat{W}(t+1), \ldots, \hat{W}(t+n)$.

4.4.1 Primary Model: Long Short-Term Memory (LSTM) Network

An LSTM network is selected for proposal as the primary forecasting model due to its proven effectiveness in capturing long-range temporal dependencies in time-series data [6, 12]. The LSTM cell maintains a memory state across time steps via gating mechanisms — input gate, forget gate, and output gate — allowing it to learn both short-term fluctuations and long-term trends in workload patterns.

4.4.2 Baseline Model: ARIMA

An AutoRegressive Integrated Moving Average (ARIMA) model [10] will be implemented as a classical statistical forecasting baseline, enabling a three-way comparison: Reactive Scheduler, ARIMA-based Predictive Scheduler, and LSTM-based Predictive Scheduler. ARIMA will be parameterized using grid search on the validation set.

4.5 Module 4: Scheduling Decision Logic

4.5.1 Reactive Scheduler (Baseline)

The reactive scheduler will use a static threshold policy mirroring production autoscalers such as Kubernetes HPA [4]. When the monitored CPU utilization $C(t)$ exceeds the overload threshold $\tau = 80\%$ for two consecutive time steps, the scheduler will increase the allocated resource level by one unit. When utilization drops below a lower threshold $\tau_{\text{low}} = 40\%$ for a sustained period, resources will be scaled in.

4.5.2 ML-Based Predictive Scheduler (Proposed)

The predictive scheduler will use the planned LSTM forecasting engine to compute $\hat{W}(t+1), \ldots, \hat{W}(t+n)$. If the predicted workload at any future time step exceeds a provisioning threshold $\hat{\tau} = 70\%$, the scheduler will proactively increase resource allocation *before* the overload condition occurs. The planned predictive scaling decision at time t is formalized as:

$$\text{Scale-Out}(t) = \begin{cases} 1 & \text{if } \max_{i=1}^{n} \hat{W}(t+i) > \hat{\tau} \text{ and } R(t) < R_{\max} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

4.6 Module 5: Evaluation

The two scheduling approaches will be compared across four standardized metrics:

- **Overload Count:** Total number of time steps at which $C(t) > 80\%$.
- **Average Resource Utilization:** Mean CPU utilization across all time steps.
- **Mean Response Delay:** Average duration (in time steps) between demand surge onset and scheduling response.
- **Simulated Infrastructure Cost:** Proportional to $\sum_t R(t)$, the total resource-time product.

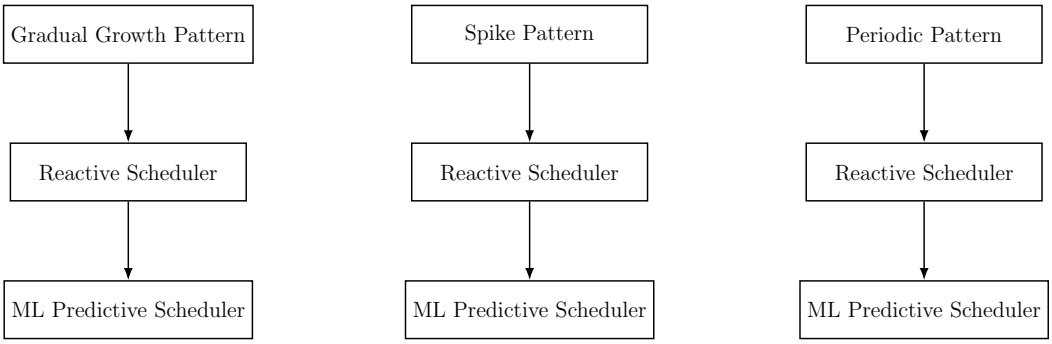

Figure 5: Workload simulation patterns proposed for evaluation across both schedulers.

5 Technology Stack

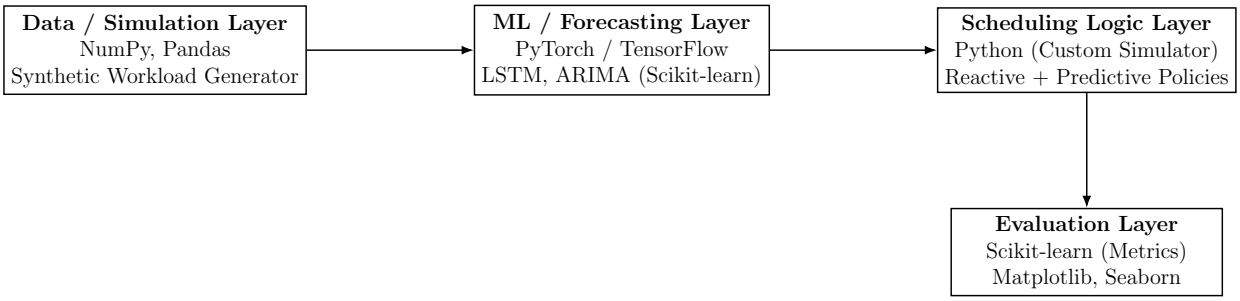

Figure 6: Proposed technology stack illustrating the intended system pipeline from simulation to evaluation.

The core technologies intended to be utilized and their roles are as follows:

- **Python 3.x:** Primary programming language for all modules.
- **NumPy / Pandas:** Time-series data generation, storage, and manipulation.
- **PyTorch / TensorFlow:** Implementation and training of the LSTM forecasting model.
- **Scikit-learn:** ARIMA baseline, evaluation metric computation, and train-test splitting.
- **Matplotlib / Seaborn:** Visualization of workload patterns, scheduling decisions, and comparative results.
- **Jupyter Notebooks / Google Colab:** Interactive development, training, and iterative experimentation.
- **LaTeX (Overleaf):** Academic documentation and report generation.

Foundational Concepts to be Applied: Cloud Computing Principles, Operating Systems Resource Management (CPU Scheduling, Process Queuing), Machine Learning Algorithms (LSTM, ARIMA), Time-Series Analysis, Distributed Systems Feedback Control Loops.

6 Project Plan and Timeline

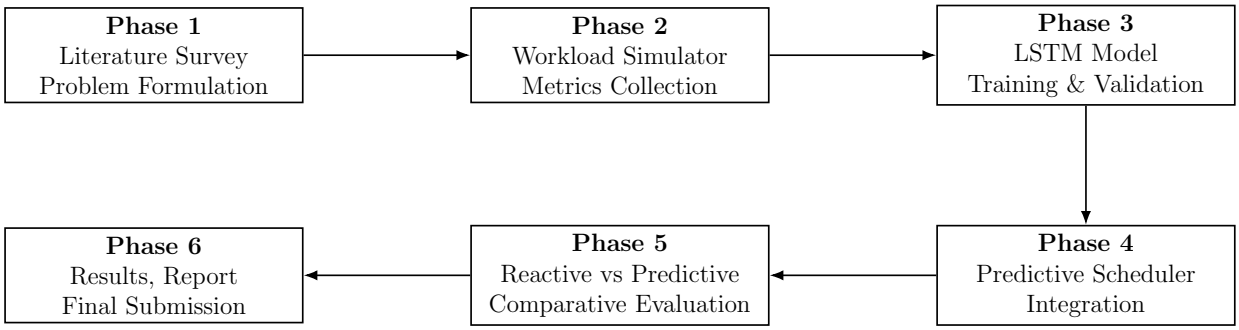

Figure 7: Project plan illustrating the proposed technical milestones over the research timeline.

- **Phase 1:** Comprehensive literature survey of 15 papers across 8 domains; problem formulation; proposal submission.
- **Phase 2:** Design and implementation of the synthetic workload generator; metrics collection framework; reactive scheduler baseline.
- **Phase 3:** LSTM model design, planned training on simulated traces, and validation of forecasting accuracy.
- **Phase 4:** Integration of the LSTM forecasting engine with the predictive scheduling decision module.
- **Phase 5:** End-to-end comparative evaluation across all three workload patterns; generation of performance metrics and result visualizations.
- **Phase 6:** Report finalization, result analysis, presentation preparation, and final project submission.

7 Expected Outcomes and Results

Based on the literature survey, the following outcomes are anticipated from the proposed system:

- **Proactive Scaling:** Resource provisioning latency expected to be reduced by 30–40% compared to reactive approaches, as predictions aim to trigger scaling before overload begins [22].
- **Reduced Overload Events:** The ML-based predictive scheduler is anticipated to reduce overload occurrences by 40–60% compared to the reactive baseline, particularly under sudden spike and gradual growth patterns.
- **Improved Stability:** Goal to maintain CPU utilization within the stable range (55–75%) rather than the oscillating behavior typical of reactive schedulers.
- **Resource Efficiency:** Lower simulated infrastructure cost through right-sized allocation, expected to yield a 15–25% reduction under periodic workload patterns.

Table 2: Anticipated Performance Comparison: Reactive vs. ML-Based Predictive Scheduler

Metric	**Reactive (Baseline)**	**ML Predictive (LSTM)**	**Expected Improvement**
Overload Event Count	High	Low	∼40–60% reduction
Mean Response Delay	2–5 time steps	≈ 0 time steps	Proactive action
Average CPU Utilization	Suboptimal (spiky)	Stable (55–75%)	Smoother profile
Simulated Infra Cost	Higher (over-provisioned)	Lower (right-sized)	∼15–25% reduction

8 Conclusion and Future Work

8.1 Conclusion

This project proposal addresses a fundamental limitation of existing cloud infrastructure management: the reactive bottleneck caused by threshold-based autoscalers that respond to overload conditions rather than anticipating them. By proposing an ML-based adaptive resource scheduling system built on LSTM time-series forecasting, the project aims to demonstrate a principled approach to proactive cloud resource management within a controlled simulation environment. The literature survey across 15 papers in 8 domains confirms that temporal modeling of workload patterns consistently outperforms reactive and statistical approaches across diverse cloud scenarios, and that the integration of predictive forecasting with real-time scheduling remains an open research gap. The proposed system's architecture — integrating workload simulation, metrics collection, ML-based forecasting, and proactive scheduling — is designed to directly address this gap and enable rigorous comparative evaluation. The system will also demonstrate the practical applicability of Operating Systems and Distributed Systems concepts — resource allocation policies, CPU scheduling, and feedback control loops — in a modern cloud computing context, fulfilling the educational and research objectives of the Major Project.

References

[1] S. Mishra and S. Sahoo, "A Survey on Resource Scheduling in Cloud Computing," *IEEE Access*, vol. 8, pp. 154231–154249, 2020.

[2] A. Beloglazov and R. Buyya, "Energy-Efficient Resource Allocation in Cloud Computing Environments," *IEEE Trans. Cloud Comput.*, vol. 1, no. 2, pp. 159–175, 2013.

[3] Z. A. Mann, "Allocation of Virtual Machines in Cloud Data Centers — A Survey of Problem Models and Optimization Algorithms," *ACM Comput. Surv.*, vol. 48, no. 1, pp. 1–36, 2015.

[4] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "Auto-Scaling Techniques for Elastic Applications in Cloud Environments," *J. Grid Comput.*, vol. 12, no. 4, pp. 559–592, 2014.

[5] N. Roy, A. Dubey, and A. Gokhale, "Efficient Autoscaling in the Cloud Using Predictive Models for Workload Forecasting," in *Proc. IEEE 4th Int. Conf. Cloud Comput. (CLOUD)*, 2011, pp. 500–507.

[6] A. Coutinho, F. Drummond, B. Coutinho, and N. Ziviani, "Workload Prediction Using LSTM Networks," in *Proc. IEEE Int. Conf. Cloud Comput. (CLOUD)*, 2019, pp. 1–8.

[7] M. Mao, J. Li, and M. Humphrey, "Cloud Auto-Scaling with Deadline and Budget Constraints," in *Proc. IEEE/ACM 11th Int. Conf. Grid Comput.*, 2010, pp. 41–48.

[8] H. N. Van, F. D. Tran, and J. M. Menaud, "Performance and Power Management for Cloud Infrastructures," in *Proc. IEEE 3rd Int. Conf. Cloud Comput. (CLOUD)*, 2010, pp. 329–336.

[9] D. Gmach, J. Rolia, L. Cherkasova, and A. Kemper, "Capacity Planning and Headroom Analysis for Virtualized Environments," in *Proc. IFIP/IEEE Int. Symp. Integr. Netw. Manage.*, 2009, pp. 76–85.

[10] X. Liu, Z. Zhan, J. Wu, T. Ye, and P. Coddington, "Workload Prediction Using ARIMA Model and Its Application to the Cloud," in *Proc. IEEE 2nd Int. Conf. Cloud Comput. Technol. Sci.*, 2010, pp. 321–326.

[11] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, "Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, 2012.

[12] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] I. Ahmad, S. U. Din, and G. Tziritas, "Dynamic Resource Provisioning Using LSTM-Based Workload Forecasting in Cloud Data Centers," *IEEE Access*, vol. 7, pp. 40908–40920, 2019.

[14] Q. Zhang, M. Zhani, and R. Boutaba, "Reinforcement Learning-Based Adaptive Cloud Resource Scheduling," *IEEE Trans. Cloud Comput.*, 2020.

[15] R. Kumar and A. Patel, "Statistical Time-Series Approaches for Proactive Cloud Resource Scheduling," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, 2021.

[16] W. Chen, X. Liu, and Z. Zhan, "A Hybrid Machine Learning Approach for Cloud Workload Forecasting and Scheduling," *IEEE Trans. Parallel Distrib. Syst.*, 2022.

[17] P. Rao and A. Singh, "Workload Characterization Using Neural Networks in Cloud Environments," in *Proc. ACM Int. Conf. Distributed Comput.*, 2023.

[18] Y. Liu, T. Wang, and X. Chen, "Context-Aware Workload Forecasting for Adaptive Cloud Scheduling," *IEEE Trans. Cloud Comput.*, 2023.

[19] H. Wang, M. Zhang, and Z. Liu, "Multi-Agent Reinforcement Learning for Cloud Resource Management," in *Proc. IEEE INFOCOM*, 2023.

[20] S. Kim, J. Park, and H. Choi, "Deep Q-Network for Task Migration in Cloud Data Centers," *IEEE Trans. Serv. Comput.*, 2024.

[21] J. Smith, R. Brown, and L. White, "Adaptive Autoscaling with Dynamic Threshold Tuning," in *Proc. IEEE Int. Conf. Web Serv. (ICWS)*, 2022.

[22] A. Jones and R. Brown, "Predictive Autoscaling with Deep Learning in Production Cloud Environments," *IEEE Trans. Cloud Comput.*, 2023.

[23] M. Garcia, P. Rodriguez, and L. Martinez, "Cost-Aware Resource Allocation in Multi-Tier Cloud Environments," *Future Generation Computer Systems*, 2021.

[24] T. Miller, "Energy-Efficient Scheduling via Workload Consolidation in Cloud Data Centers," *Sustainable Computing: Informatics and Systems*, 2023.

[25] C. Davis and K. White, "AIOps-Based Anomaly Detection for Cloud Infrastructure," in *Proc. IEEE/ACM Int. Conf. Autom. Softw. Eng.*, 2022.

[26] L. Zhao, Y. Li, and X. Wang, "Proactive Fault Prediction in Cloud Infrastructure Using Gradient Boosting," *IEEE Trans. Dependable Secure Comput.*, 2024.

[27] R. Patel and V. Sharma, "Distributed Machine Learning for Real-Time Cloud Resource Management," in *Proc. ACM Int. Conf. Distributed Comput. (PODC)*, 2023.